REST-Service
Dr. F.-K. Koschnick, Sybit GmbH

Copyright © 2024 Dr. F. K. Koschnick, Sybit GmbH. All rights reserved.

Representational State Transfer (REST)

Roy Fielding: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm
Dissertation im Jahr 2000

REST beschreibt einen Architektur-Stil für die Kommunikation bei Verteilten Systemen, insbesondere bei Webservices. Mit REST-Service ist im Folgenden ein Webservice gemeint, der die Bedingungen für REST implementiert.

Die Kernidee bei REST ist das Konzept der Ressource. Diese Ressource wird über einen Medientyp (Repräsentation) dargestellt. Beispiele für Medientypen sind JSON oder XML.

Ein Bespiel für die URL einer Ressource, die einen Datensatz für einen Kunden mit der ID=33245 umfasst, ist: https://myServer/customers/33245. Wenn alle Kunden abgefragt werden sollen, würde die URL https://myServer/customers sein.

REST: Die sechs Bedingungen

1. Vereinheitlichtes Interface
   - Adressierbarkeit
   - Entkopplung von Ressourcen und Repräsentation
   - Standard-HTTP-Methoden
2. Zustandslos
3. Cacheable
4. Client-Server
5. Mehrschichtige Systeme
6. Code on Demand (optional, betrachten wir hier nicht)

REST: Vereinheitlichtes Interface (1)

Adressierbarkeit:
Eindeutige Adresse einer Ressource mit Hilfe eines Unique Ressource Identifiers (URI)
Beispiel : http://141.37.122.35:8080/easynotes/api/notes/1

Entkopplung von Ressourcen und Repräsentation:
Beispiel: Die Ressource ist ein Java-Objekt, das aus der Datenbank „geholt“ wurde (z.B.: JPA),
Während die Repräsentation ein JSON-String ist

```
{
    "id": 1,
    "title": "Bemerkung1",
    "content": "Inhalt1",
    "created": "2018-07-23T17:49:52.000+0000",
    "updated": "2018-07-24T13:01:09.000+0000"
}
```

Standard-HTTP-Methoden:
GET: Lesen von Daten (idempotent (gleiches Ergebnis, ob der Vorgang ein oder mehrere Male wiederholt wird)
POST: Erzeugen einer neuen Ressource (nicht idempotent)
PUT: Verändern einer Ressource (idempotent, soll auf alle Fälle idempotent implementiert sein)
DELETE: Löschen einer Ressource (bedingt idempotent, Response Code ändert sich nach der ersten Ausführung)

REST: Vereinheitlichtes Interface (1), HTTP-Methoden

Da das HTTP-Protokoll im Internet weit verbreitet ist, werden REST-Services in der Regel mit den Standard-HTTP-Methoden GET, POST, PUT und DELETE implementiert:

PATCH: https://blog.postman.com/what-are-http-methods/
PATCH is used for **modify** capabilities. The PATCH request only needs to contain the changes to the resource, not the complete resource.

HTTP Methode	CRUD	Gesamte Collection (z.B.: /customers)	Specifisches Item (z.B.: /customers/{id})
POST	Create	201 (Created), 'Location' header with link to /customers/{id} containing new ID. *)	404 (Not Found), 409 (Conflict) if resource already exists.
GET	Read	200 (OK), list of customers. Use pagination, sorting and filtering to navigate big lists.	200 (OK), single customer. 404 (Not Found), if ID not found or invalid.
PUT	Update	405 (Method Not Allowed), unless you want to update/replace every resource in the entire collection.	200 (OK) or 204 (No Content). 404 (Not Found), if ID not found or invalid.
DELETE	Delete	405 (Method Not Allowed), unless you want to delete the whole collection—not often desirable.	200 (OK). 404 (Not Found), if ID not found or invalid.

*) `response.setHeader("Location", "/customers/4711");`

Zustandslos (2)

Das zweite wichtige Prinzip von REST besteht darin, dass die Kommunikation immer zustandslos sein muss:

- keine mit Sessions und Cookies realisierten Benutzersitzungen
- bei jeder Anfrage werden sämtliche erforderlichen Informationen erneut mitgesendet.

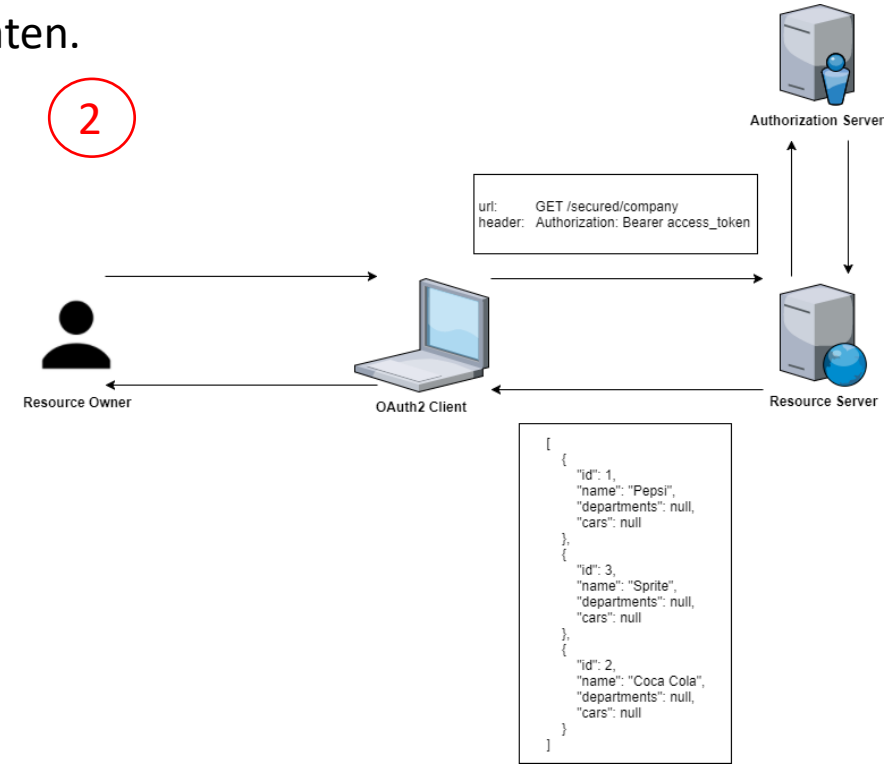
Der Vorteil (und Hauptgrund) der Zustandslosigkeit von REST-Services ist die einfache Skalierbarkeit. Gibt es nämlich keine Sessions, lassen sich mehrere Anfragen eines Clients problemlos auf verschiedene Server verteilen.

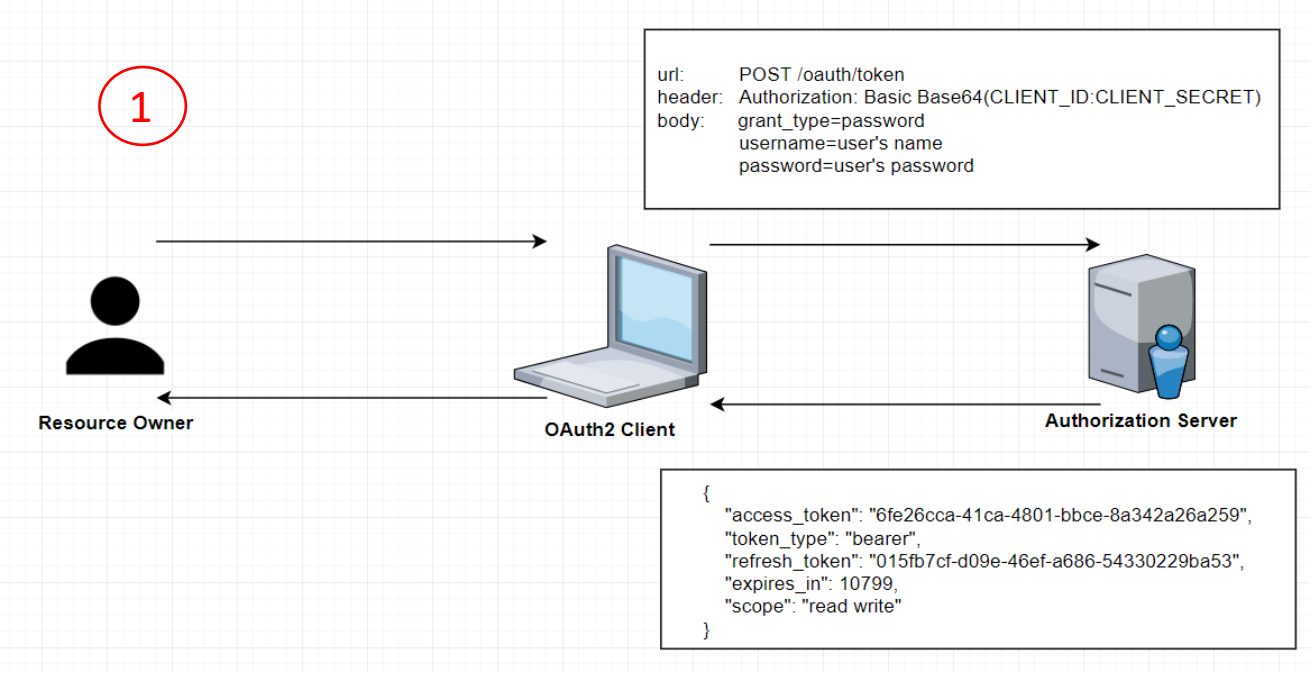
Absichern eines REST-Services (Zustandslos (2))

https://dzone.com/articles/secure-spring-rest-with-spring-security-and-oauth2

REST-Service

1. User loggt sich über einen Autorisierungs-Server in eine Applikation ein und bekommt einen Token.
2. Der Token wird bei einer Anfrage an den REST-Service geschickt. Der REST-Service fragt am Autorisierungs-Server mit dem Token an und liefert, wenn der Autorisierungs-Server die Rechte frei gibt, die Daten.





REST: Cacheable (3)

REST-Services müssen das Caching am Client unterstützen, damit nicht unnötig viele Anfragen gestellt werden. Um ein Cache-Management durchzuführen, so dass weder zu viele unnötige Anfragen gestellt werden, aber auch nicht veraltete Daten im Cache gehalten werden, kann man im Response-Header das Expires und das Cache-Control setzen.

Beispiele:
Expires: Wed, Oct 24 2018 9:20:49 AM UTC

Cache-Control: no-cache
Cache-Control: public/private
Cache-Control: max-age=<seconds>

```
response.setHeader("Cache-Control", "max-age=3600");
```

REST: Cacheable (3) -> Header

Elements Console Sources Network Performance Memory Application

View: Group by frame Preserve log Disable cache Offline Online

Filter Hide data URLs All XHR JS CSS Img Media Font Doc WS Manifest Other

10 ms 20 ms 30 ms 40 ms 50 ms 60 ms 70 ms 80 ms 90 ms 100 ms 110

Name

notes

1 requests | 694 B transferred | Fi...

Headers Preview Response Timing

General

Request URL: `http://delia:8080/easynotes/api/notes`

Request Method: `GET`

Status Code: 200

Remote Address: `192.168.1.247:8080`

Referrer Policy: `no-referrer-when-downgrade`

Response Headers view source

Cache-Control: `no-cache`

Content-Type: `application/json;charset=UTF-8`

Date: `Mon, 05 Nov 2018 15:13:34 GMT`

Transfer-Encoding: `chunked`

Request Headers view source

REST: Client-Server (4)

Die einheitliche Schnittstelle, die der REST-Service anbietet, sorgt für eine Trennung der Belange (Separation of Concern):

- Client und Server können unterschiedliche Betriebssysteme haben
- Client-Software und Server-Software können in unterschiedlichen Programmiersprachen geschrieben sein
- Solange das Interface des Servers stabil bleibt, kann dessen Implementierung beliebig verändert werden, ohne das der Client das merkt.
- Der Server muss sich nicht mit dem „Zustand“ des Users befassen (z.B.: eingeloggt oder nicht) -> Skalierbarkeit

REST: Mehrschichtige Systeme (5)

- Ein Client kann normalerweise nicht feststellen, ob er direkt mit dem Endserver oder einem Vermittler auf dem Weg verbunden ist.
- Loadbalancer können die Last auf mehrere Server verteilen und/oder gemeinsam genutzte Caches bereitstellen
- Security-Schichten (siehe oauth2-Server) können einen Service absichern, ohne dessen Zustandslosigkeit zu zerstören

Beispiele für das Interface

Gutes Beispiel: /users/12345
Schlechtes Beispiel: /api?type=user&id=12345

Gutes Beispiel: /customers/33245/orders/4711/positions/1
Schlechtes Beispiel: /customer/33245/order/4711/position/1

Listen durch Plural ausdrücken, z.B.: customers
Nicht customer_list
Pfade in URL so kurz wie möglich halten und am besten nur Kleinbuchstaben verwenden

Beispiele für das Interface

HTTP GET http://api.example.com/device-management/managed-devices //Get all devices

HTTP POST http://api.example.com/device-management/managed-devices //Create new Device

HTTP GET http://api.example.com/device-management/managed-devices/{id} //Get device for given Id

HTTP PUT http://api.example.com/device-management/managed-devices/{id} //Update device for given Id

HTTP DELETE http://api.example.com/device-management/managed-devices/{id} //Delete device for given Id

Nachtrag zum Interface: Response Code

Response Code	Beschreibung
200 OK	Allgemeiner Erfolgs-Status-Code (z.B. bei get)
201 CREATED	Erfolgreiche Erzeugung/Modifikation einer Ressource durch POST, PUT
204 NO CONTENT	Oft genutzt bei DELETE oder PUT, wenn die Ressource nicht vorhanden ist, aber Erfolg
400 BAD REQUEST	z.B. fehlende Daten im Request
401 UNAUTHORIZED	Fehlendes oder ungültiges Autorisierungstoken
403 FORBIDDEN	Wenn die Operation nicht ausgeführt werden darf, z.B. weil die Ressource nicht (mehr) verfügbar ist
404 NOT FOUND	Ressource nicht gefunden, evtl. zum Maskieren von 401 und 403
405 METHOD NOT ALLOWED	Z.B. wenn POST nicht verfügbar, weil API keine Erzeugung einer Ressource unterstützt
409 CONFLICT	Duplikate oder Parent-Element kann nicht gelöscht werden, weil kein Kaskadierendes Löschen implementiert ist.
500 INTERNAL SERVER ERROR	Muss abgefangen werden, wenn ein 500er „nach draußen“ geht, ist das immer ein Bug.

REST-Service mit Spring-Boot


```
...
@RestController
public class ProjectRestController {

    private static final org.slf4j.Logger LOG = LoggerFactory.getLogger(ProjectRestController.class);
    private ProjectService projectService;
    private SessionService sessionService;

    @Autowired
    public ProjectRestController(ProjectService projectService, SessionService sessionService) {
        this.projectService = projectService;
        this.sessionService = sessionService;
    }

    @RequestMapping(value = "/projects", method = RequestMethod.GET, produces = MediaType.APPLICATION_JSON_VALUE)
    public List<Project> getAllProjects(HttpServletRequest request, HttpServletResponse response) { //Map<String, Object> model,
        LOG.debug("--> getAllProjects");
        HttpSession session = request.getSession();
        if(sessionService.isLoggedIn(session)) { //this is not compliant to REST, sessions are not allowed, but we do not use an aut
            response.setHeader("Cache-Control", "no-cache");
            LOG.debug("<-- getAllProjects");
            return projectService.getByUserId(sessionService.getUserId(session)); //ArrayList implements Serializable, therefore th
        } else {
            response.setStatus(401); //http status unauthorized
            LOG.debug("<-- getAllProjects");
            return new ArrayList<>(); //return empty List
        }
    }
```

Annotation: RestController

Autowire, wie schon gehabt

RequestMapping wie gehabt, es wird nur JSON zurückgegeben

Hier wird direkt eine ArrayList zurückgegeben (kein View)

Beispiel einer REST-Methode für die Ressource Projekt

Eine URL für diese REST-Methode wäre z.B. …/projects/1

```
@RequestMapping(value = "/projects/{id}", method = RequestMethod.GET, produces = MediaType.APPLICATION_JSON_VALUE)
public Project getProjectById(@PathVariable(value = "id") Long projectId, HttpServletResponse response) {
  response.setHeader("Cache-Control", "no-cache");
  try {
    return projectService.getById(projectId);
  } catch (ResourceNotFoundException e) {
    LOG.debug("getProjectById",e);
    response.setStatus(404);
    return null;
  }
}
```

Alternative zu response.setStatus(404) wäre Annotation an Exception:

@ResponseStatus(value = HttpStatus.***NOT_FOUND***)

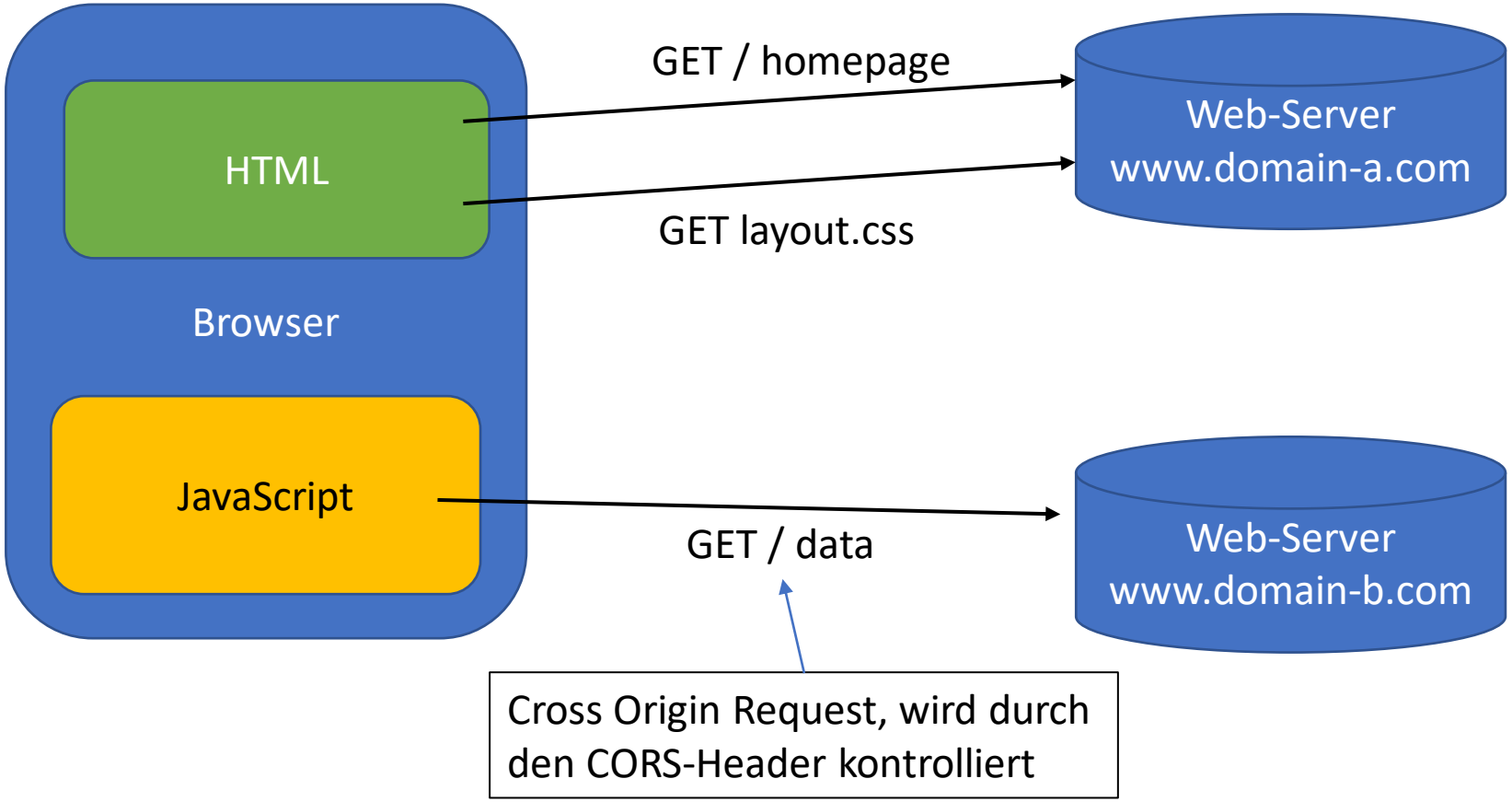
Cross-Origin Resource Sharing (CORS)
https://developer.mozilla.org/de/docs/Web/HTTP/CORS

Cross-Origin Resource Sharing (CORS) ist ein Mechanismus, der zusätzliche HTTP-Header verwendet, um einem Browser mitzuteilen, dass eine Webanwendung, die an in einer Domäne ausgeführt wird, über die Berechtigung verfügt, auf ausgewählte Ressourcen von einem Server aus einer anderen Domäne zuzugreifen. Eine Webanwendung stellt eine ursprungsübergreifende HTTP-Anforderung aus, wenn sie eine Ressource anfordert, die eine andere Domäne, ein anderes Protokoll oder einen anderen Port (Ursprung) hat.

Ein Beispiel für eine Anfrage: Der Frontend-JavaScript-Code für eine Webanwendung, die von http://domain-a.com bereitgestellt wird, verwendet XMLHttpRequest, um eine Anfrage für http://api.domain-b.com/projects zu stellen.

Aus Sicherheitsgründen beschränken Browser herkunftsübergreifende HTTP-Anforderungen, die innerhalb von Skripts initiiert werden. Beispielsweise folgen XMLHttpRequest und die Fetch-API der Richtlinie mit demselben Ursprung. Dies bedeutet, dass eine Webanwendung, die diese APIs verwendet, nur HTTP-Ressourcen von demselben Ursprung anfordern kann, aus dem die Anwendung geladen wurde, es sei denn, die Antwort des anderen Ursprungs enthält die richtigen CORS-Header.

Cross-Origin Resource Sharing (CORS)



Was ist, wenn Sie im Browser zwei Tabs offen haben: mit einem machen Sie Online-Banking, mit dem anderen haben Sie eine zweifelhafte Seite geöffnet?

Sie wollen nicht, dass die zweifelhafte Seite über Cross-Origin-Requests im Hintergrund Banktransaktionen über die geöffnete Banking-Session ausführt!

Hoffentlich hat die Online-Banking-Seite durch CORS den Zugriff beschränkt!!!

Cross-Origin Resource Sharing (CORS)

CORS-Header (https://spring.io/blog/2015/06/08/cors-support-in-spring-framework)

Cross-Origin Resource Sharing (CORS) ist ein Mechanismus, der Webbrowsern oder auch anderen Webclients Cross-Origin-Requests ermöglicht. Zugriffe dieser Art sind normalerweise durch die Same-Origin-Policy (SOP) untersagt. CORS ist ein Kompromiss zugunsten größerer Flexibilität im Internet unter Berücksichtigung möglichst hoher Sicherheitsmaßnahmen. Der Server sendet einen Header im Response-Objekt, dass dem Browser den Zugriff erlaubt.

…

```
@RestController
@RequestMapping("/api")
public class NoteController {

  @Autowired
  NoteService noteService;

// Get All Notes
  @CrossOrigin(origins = "*") //CORS (Cross-origin resource sharing) for all sites
  @GetMapping("/notes")
  public List<Note> getAllNotes(HttpServletResponse response) {
  response.setHeader("Cache-Control", "no-cache");
    return noteService.getAllNotes();
  }
…
```

Setzen des Response-Headers für CrossOrigin. Hier wird Zugriff von allen Sites erlaubt.

Header im Response:

Access-Control-Allow-Origin: *

Möglich wäre beispielsweise auch:
Access-Control-Allow-Origin: https://foo.example

REST-Tutorial

https://www.restapitutorial.com/